

Dobot ROS2 Bridge

Isaac Sim Extension – Technische Dokumentation
Bidirektionaler Digital-Twin-Kopplungsrahmen für den
Dobot Magician in NVIDIA Isaac Sim 5.1

Tobias Küster

26. Juni 2026

Inhaltsverzeichnis

1	Übersicht und Zielsetzung	4
1.1	Zielsetzung	4
2	Systemumgebung und Softwarestapel	4
2.1	Hardware	4
2.2	NVIDIA-Treiber und CUDA	4
2.3	NVIDIA Isaac Sim	5
2.4	ROS2-Integration in Isaac Sim	5
2.5	Pythonpakete der Projekt-Entwicklungsumgebung	6
2.6	Laufzeitabhängigkeiten (Isaac-Sim-Umgebung)	6
2.7	Systemanforderungen (Zusammenfassung)	6
3	Datei- und Paketstruktur	7
3.1	Quellcode-Dateien	7
3.2	Extension-Manifest (<code>extension.toml</code>)	7
3.3	Installations-Hilfsskript (<code>install_extension.py</code>)	7
3.4	USD- und Robotermodell-Dateien	8
4	Systemarchitektur	8
4.1	Schichtenmodell	8
4.2	Hauptklassen	8
4.3	Datenfluss	9
4.3.1	Richtung 1: Echter Roboter → Simulation (Digital Twin)	9
4.3.2	Richtung 2: Simulation / UI → Echter Roboter	9
5	Lazy-Import-System	9
6	Kinematisches Modell	10
6.1	Koordinatenkonventionen (pydobot)	10
6.2	Isaac-Sim-Gelenkstruktur	10
6.3	Einheitenstrategie	10
6.4	Berechnungsformeln	11
6.4.1	joint_1 – Gierwinkel mit festem Offset	11
6.4.2	joint_2 – Direkte Übernahme	11
6.4.3	joint_5 – Parallelogramm-Kompensation	11

6.4.4	joint_6 – Endeffektor-Vertikalisierung	11
6.5	Vollständige Kinematik in <code>update_step()</code>	12
7	Klasse DobotROSNode	12
7.1	Initialisierung (<code>__init__</code>)	12
7.2	Methoden-Referenz	13
7.3	Fehlerbehandlung und Robustheit	13
8	Servo-GP3-Ansteuerung	14
8.1	Hardware-Hintergrund	14
8.2	Duty-Cycle-Berechnung	14
8.3	Protokoll-Implementierung	14
8.4	J1-Weltwinkel-Kompensation	14
9	Dynamic-Control-Kopplung	15
9.1	Initialisierungsproblem	15
9.2	Lazy-Initialisierung	15
9.3	DOF-Handles abrufen	15
9.4	Gelenkwinkel setzen – <code>_drive_joints()</code>	15
9.5	Artikulations-USD-Pfad	16
10	Teach-in-System (Pick & Place)	16
10.1	Datenstruktur der Wegpunkte	16
10.2	Aufzeichnung	16
10.3	Wiedergabe-Thread	16
10.4	Bewegungsablauf pro Wegpunkt	16
10.5	Rückkehr zur Nullposition	17
10.6	Notstopp	17
11	Ziel-Würfel-Tracking	17
11.1	Würfel erstellen (<code>_spawn_target_cube()</code>)	17
11.2	Tracking-Schleife (<code>_tracking_loop_thread()</code>)	17
11.3	Würfel entfernen (<code>_despawn_target_cube()</code>)	18
12	Klasse DobotBridgeExtension	18
12.1	Zustandsvariablen	18
12.2	Lebenszyklusmethoden	18
12.3	Frame-Update-Ablauf (<code>_on_update()</code>)	19
12.4	Ressourcenfreigabe (<code>_cleanup()</code>)	19
13	UI-Fenster	20
13.1	Tab 0: Steuerung	20
13.2	Tab 1: Pick & Place	20
13.3	Status-Dashboard (immer sichtbar)	21
14	Threading-Modell	21
15	ROS2-Integration	21
15.1	Node-Konfiguration	21
15.2	Nachrichtenstruktur	21
15.3	ROS2-Kontext-Verwaltung	22

16 Konstanten (constants.py)	22
16.1 Farbwerte (ARGB-Hex)	22
16.2 Nullposition und Servo	22
16.3 USD-Pfade	22
17 URDF-Robotermodell (load_dobot.urdf)	22
17.1 Gelenkstruktur	23
17.2 Körper (Links)	23
18 Installation und Inbetriebnahme	23
18.1 Erstinstallation	23
18.2 Betrieb Schritt für Schritt	23
18.3 Teach-in-Workflow	23
19 Bekannte Einschränkungen	24
20 Entwicklungsverlauf	24

1 Übersicht und Zielsetzung

Die Extension `dobot_ros` implementiert eine **bidirektionale Echtzeit-Kopplung** zwischen einem realen **Dobot Magician**-Roboterarm und einem **NVIDIA Isaac Sim 5.1**-Simulationsmodell. Sie stellt damit einen vollständigen Digital-Twin-Kopplungsrahmen dar, der es ermöglicht, den physischen Roboter in einer virtuellen Simulationsumgebung zu spiegeln und umgekehrt Steuerbefehle aus der Simulation an die reale Hardware zu senden.

1.1 Zielsetzung

- **Digital Twin (Richtung 1 – Real → Sim):** Der echte Roboter bewegt sich, und das Simulationsmodell folgt in Echtzeit. Gelenkwinkel werden per USB über die `pydobot`-Bibliothek gelesen, kinematisch transformiert und über die Isaac-Sim-Dynamic-Control-API auf die Simulationsgelenkstruktur übertragen.
- **Fernsteuerung (Richtung 2 – Sim → Real):** Über das UI-Panel werden Jog-Befehle, Vertikalbewegungen, Vakuumgreifer-Schaltung, RC-Servo-Ansteuerung sowie automatisierte Pick-&-Place-Sequenzen an den echten Roboter gesendet.
- **ROS2-Integration:** Die Extension startet intern einen ROS2-Node in Isaac Sim und publiziert Gelenkzustände auf dem Topic `/joint_states` (Typ: `sensor_msgs/msg/JointState`). Externe ROS2-Knoten (z. B. MoveIt!) können dieses Topic abonnieren und die aktuelle Kinematik in Echtzeit empfangen.
- **Interaktives Teach-in:** Bewegungsabläufe werden durch manuelles Verfahren des Roboters aufgezeichnet und anschließend vollautomatisch mit Sicherheitslogik wiedergegeben.
- **Würfel-Tracking (Ziel-Tracking):** Ein interaktiv in der Isaac-Sim-Szene verschiebbarer Ziel-Würfel dient als Bewegungssollwert für den echten Roboterarm.

2 Systemumgebung und Softwarestapel

Dieser Abschnitt dokumentiert vollständig die Hardware- und Softwareumgebung, in der die Extension entwickelt und betrieben wird.

2.1 Hardware

Komponente	Spezifikation
CPU	Intel Core i7-13620H (13. Generation, Hybrid-Architektur)
RAM	32 GB DDR5
GPU	NVIDIA GeForce RTX 4070 Laptop GPU, 8 GB VRAM (WDDM)
Betriebssystem	Windows 11 Home, Build 26200
Roboter	Dobot Magician, USB-Verbindung über COM3

2.2 NVIDIA-Treiber und CUDA

Software	Version
NVIDIA GeForce Treiber (Game Ready)	592.27 (2. März 2026)
CUDA Version (maximale Unterstützung durch Treiber)	13.1
CUDA Toolkit (installiert)	12.1.66
nvcc (CUDA Compiler Driver)	12.1, Build 32415258
CUDA-Pfad	C:\Program Files\NVIDIA GPU Computing Tool
Nsight Compute	2023.1.0
Nsight Systems	2022.4.2 / 2023.1.2
NVIDIA Nsight Visual Studio Edition	25.4.0.25287

Treiber-Mindestanforderung von Isaac Sim 5.1: Laut der mitgelieferten Datei `kit/driver-requirements.txt` werden NVIDIA-Treiber unterhalb von Version **537.58** (Windows) vom RTX-Renderer blockiert. Der installierte Treiber 592.27 liegt deutlich darüber und ist vollständig kompatibel.

2.3 NVIDIA Isaac Sim

Parameter	Wert
Isaac Sim Version	5.1.0-rc.19+release.26219.9c81211b.gl
Installationspfad	C:\NVIDIA_Isaac_Sim
Interne Python-Version	3.11.13 (gebündeltes Kit-Python)
Python-Executable	C:\NVIDIA_Isaac_Sim\python.bat
Kit-Executable	C:\NVIDIA_Isaac_Sim\kit\kit.exe

Isaac Sim bringt eine **eigene, isolierte Python-3.11.13-Laufzeitumgebung** mit. Diese ist vollständig von einer etwaigen System-Python-Installation getrennt. Alle Extension-Skripte laufen in dieser gebündelten Python-Umgebung – **nicht** in der System-Python-Installation (hier: Python 3.14.3 im Projekt-`.venv`).

2.4 ROS2-Integration in Isaac Sim

ROS2 wird nicht separat installiert. Isaac Sim 5.1 enthält eine eingebettete ROS2-Bridge-Extension unter:

C:\NVIDIA_Isaac_Sim\exts\isaacsim.ros2.bridge

Diese Bridge stellt ROS2 Humble **und** ROS2 Jazzy als vorkompilierte Shared Libraries bereit:

Parameter	Wert
Unterstützte ROS2-Distributionen	Humble (Standard) / Jazzy
RMW-Implementierung (Middleware)	rmw_fastrtps_cpp (Fast DDS)
Standard-Distro-Umgebungsvariable	ROS_DISTRO=humble
Library-Pfad (Humble)	isaacsim.ros2.bridge\humble\lib
rclpy-Pfad	isaacsim.ros2.bridge\rclpy

Die Umgebungsvariablen werden beim Start über `setup_ros_env.bat` gesetzt, das in `isaac-sim-dobot.bat` (Abschnitt ??) eingebunden ist.

2.5 Pythonpakete der Projekt-Entwicklungsumgebung

Das Projekt-`.venv` (Python 3.14.3, `dobot_ros/.venv`) dient **ausschließlich** für Entwicklungswerkzeuge (z. B. LaTeX-Build-Skript) und enthält nur:

Paket	Version
<code>paho-mqtt</code>	2.1.0
<code>pip</code>	26.1

Die eigentlichen Laufzeitabhängigkeiten (`pydobot`, `pyserial`, `rclpy`, `omni.*`) werden **innerhalb der Isaac-Sim-Python-Umgebung** installiert bzw. sind dort bereits enthalten.

2.6 Laufzeitabhängigkeiten (Isaac-Sim-Umgebung)

Paket	Version	Bereitstellung
<code>pydobot</code>	≥ 1.3	via <code>omni.kit.pipapi</code> (Install-Button)
<code>pyserial</code>	≥ 3.5	via <code>omni.kit.pipapi</code> (Abhängigkeit)
<code>rclpy</code>	Humble-Build	eingebettet in Isaac-Sim-Bridge
<code>sensor_msgs</code>	Humble-Build	eingebettet in Isaac-Sim-Bridge
<code>omni.ui</code>	Kit-intern	bereits in Isaac Sim vorhanden
<code>omni.ext</code>	Kit-intern	bereits in Isaac Sim vorhanden
<code>omni.isaac.dynamic_control</code>	Kit-intern	bereits in Isaac Sim vorhanden
<code>omni.kit.pipapi</code>	Kit-intern	bereits in Isaac Sim vorhanden

2.7 Systemanforderungen (Zusammenfassung)

Komponente	Mindestanforderung / empfohlene Version
NVIDIA Isaac Sim	5.1.0 oder neuer
Isaac Sim Python	3.11 (intern, kein manueller Eingriff)
NVIDIA Treiber (Windows)	≥ 537.58 , empfohlen: 592.27+
CUDA Toolkit	12.1 oder neuer (für GPU-Physik)
GPU	NVIDIA RTX-fähige GPU, mind. 8 GB VRAM
RAM	mind. 16 GB (empfohlen 32 GB)
Betriebssystem	Windows 11 (64-Bit)
ROS2	Humble oder Jazzy (embedded, kein Standalone-Install)
Dobot Magician	USB-Verbindung, bekannter COM-Port

3 Datei- und Paketstruktur

3.1 Quellcode-Dateien

Datei	Beschreibung
<code>extension.py</code>	Hauptlogik (1536 Zeilen): ROS-Node, Kinematik, DC-Kopplung, UI, Teach-in, Würfel-Tracking
<code>constants.py</code>	Zentrale Farbwerte, USD-Pfad des Roboters, Nullposition, Servo-Konfiguration
<code>__init__.py</code>	Exportiert <code>DobotBridgeExtension</code> als Extension-Einstiegspunkt
<code>extension.toml</code>	Isaac-Sim-Manifest: Titel, Version, Python-Modul
<code>install_extension.py</code>	Hilfsskript: Kopiert Extension in den Omniverse-Extensions-Ordner
<code>set_extension_path.ps1</code>	PowerShell-Skript: Setzt <code>OMNI_KIT_EXTENSION_PATH</code>
<code>build_docs.py</code>	Erstellt <code>documentation.pdf</code> aus LaTeX
<code>build_docs.bat</code>	Windows-Batch-Wrapper für <code>build_docs.py</code>
<code>documentation.tex</code>	Diese Dokumentation (LaTeX-Quelle)

3.2 Extension-Manifest (`extension.toml`)

Die Datei `extension.toml` ist die **Visitenkarte** der Extension: Sie teilt dem Isaac-Sim-Extension-Manager mit, wie die Extension heißt, welche Version sie hat und welches Python-Modul als Einstiegspunkt dient.

Häufigste Fehlerquelle beim Laden der Extension: Wird eine Extension im Extension-Manager nicht gefunden oder lässt sich nicht korrekt laden, liegt das Problem fast immer in `extension.toml` — entweder stimmt der Dateinhalt (Modulname, Pfad) nicht, oder der Ordner, der `extension.toml` enthält, ist im Extension-Manager nicht als Suchpfad eingetragen.

```

1 [package]
2 title      = "Dobot Digital Twin"
3 description = "ROS 2 Treiber und UI fuer den Dobot"
4 version    = "1.0.0"
5 authors    = ["Tobi"]
6
7 [dependencies]
8 "omni.kit.uiapp" = {}
9 "omni.ui"        = {}
10
11 [[python.module]]
12 name = "dobot_ros"
13 path = "."

```

Listing 1: `extension.toml`

Das Modul `dobot_ros` wird aus dem Extension-Stammverzeichnis geladen; `__init__.py` exportiert `DobotBridgeExtension`, die von Isaac Sim als `omni.ext.IExt`-Implementierung erkannt wird.

3.3 Installations-Hilfsskript (`install_extension.py`)

```

1 # Standard-Installation in Omniverse-Roaming-Verzeichnis:
2 python install_extension.py
3
4 # Benutzerdefiniertes Zielverzeichnis:
5 python install_extension.py --dest "C:\MeinPfad" --name dobot_ros
6
7 # Vorschau ohne Dateikopie:
8 python install_extension.py --dry-run

```

Listing 2: install_extension.py – Verwendung

Standardmäßig wird die Extension nach %APPDATA%\NVIDIA Corporation\Omniverse\Extensions kopiert. Aus dem Quellbaum werden ausgeschlossen: .venv, .git, __pycache__, *.pyc/pyo/pyd, kompilierte LaTeX-Artefakte.

3.4 USD- und Robotermodell-Dateien

Datei	Beschreibung
load_dobot.urdf	URDF-Roboterbeschreibung (162 Zeilen): Gelenke, Massen, Kollisionsgeometrie
load_dobot/load_dobot.usd	USD-Wurzeldatei, führt alle Konfigurationsschichten zusammen
.../load_dobot_base.usd	Mesh- und Kollisionsgeometrie aller Roboterglieder (4,2 MB)
.../load_dobot_robot.usd	Artikulationsroot, Gelenkeigenschaften, IsaacRobotAPI-Schema
.../load_dobot_physics.usd	Physikparameter: Massen, Trägheitstensoren, Dämpfung, Steifigkeit
.../load_dobot_sensor.usd	Sensor-Platzhalter (Erweiterungspunkt für zukünftige Sensoren)
meshes/*.STL, *.dae	3D-Netz-Dateien für alle Roboterglieder

4 Systemarchitektur

4.1 Schichtenmodell

Die Extension ist in vier klar getrennte Schichten gegliedert:

Schicht	Klassen / Komponenten	Technologie
UI-Schicht	DobotBridgeExtension: Fenster, Tabs, Callbacks	omni.ui
Simulations-Schicht	Dynamic-Control-API, USD-Stage, omni.timeline	omni.isaac.*
Middleware-Schicht	ROS2-Node, Publisher, /joint_states	rclpy (Humble)
Hardware-Schicht	DobotROSNode: Pose-Abfrage, Bewegungsbefehle	pydobot, pyserial

4.2 Hauptklassen

DobotROSNode kapselt die serielle USB-Verbindung zum echten Dobot-Roboter über pydobot sowie den ROS2-Node und den JointState-Publisher. Enthält alle Hardware-nahen Methoden: Pose-Abfrage, Kinematikberechnung, Bewegungsbefehle, Vakuumgreifer, RC-Servo-Ansteuerung und Teach-in-Sequenzsteuerung.

DobotBridgeExtension Subklasse von `omni.ext.IExt`. Verwaltet das Omniverse-UI-Fenster mit zwei Tabs, die Isaac-Sim-Dynamic-Control-Kopplung, den Frame-Update-Loop sowie die Hintergrundthreads für Teach-in-Wiedergabe und Würfel-Tracking.

4.3 Datenfluss

4.3.1 Richtung 1: Echter Roboter → Simulation (Digital Twin)

1. Pro Frame ruft `DobotBridgeExtension._on_update()` die Methode `DobotROSNode.update_step()` auf.
2. `update_step()` liest via `device.pose()` die aktuelle Pose: $(x, y, z, r, j_1, j_2, j_3, j_4)$ in mm und Grad.
3. Kinematische Transformation (Abschnitt 6) liefert Isaac-Sim-kompatible Gelenkwinkel.
4. Konvertierung nach Radian; Publikation als `JointState` auf `/joint_states`.
5. `_drive_joints()` setzt Winkel über DC-API auf die Isaac-Sim-Artikulation.
6. Status-Dashboard im UI aktualisieren.

4.3.2 Richtung 2: Simulation / UI → Echter Roboter

1. Nutzer betätigt eine Steuertaste (z. B. $X + 5\text{ mm}$).
2. UI-Callback wird ausgelöst (z. B. `_on_jog_clicked(5.0, 0.0)`).
3. `DobotROSNode.jog(dx, dy)` wird aufgerufen.
4. Aktuelle Pose einlesen, Delta addieren, `move_to()` an Dobot senden.
5. Nächster Frame liest neue Position – Regelkreis geschlossen.

5 Lazy-Import-System

ROS2 (`rclpy`) und `pydobot` werden nicht beim Extension-Start importiert, sondern erst beim ersten Klick auf *Connect*. Dieser Mechanismus verfolgt zwei Ziele:

1. Die Extension lädt fehlerfrei, auch wenn ROS2 oder `pydobot` noch nicht installiert sind.
2. Teure Import-Operationen verzögern den Isaac-Sim-Start nicht.

Einmal erfolgreich geladene Module werden in Modul-globalen Variablen gecacht:

```

1 _rclpy      = None      # rclpy-Modul
2 _Node       = None      # rclpy.node.Node
3 _JointState = None      # sensor_msgs.msg.JointState
4 _Dobot      = None      # pydobot.Dobot
5
6 def _try_import_ros():
7     global _rclpy, _Node, _JointState
8     if _rclpy is not None:
9         return True      # Bereits gecacht
10    try:
11        import rclpy
12        from rclpy.node import Node

```

```

13     from sensor_msgs.msg import JointState
14 except Exception:
15     return False          # ROS2-Bridge nicht verfuegbar
16 _rclpy, _Node, _JointState = rclpy, Node, JointState
17 return True
18
19 def _try_import_dobot():
20     global _Dobot
21     if _Dobot is not None:
22         return True
23     try:
24         from pydobot import Dobot
25     except Exception:
26         return False      # pydobot nicht installiert
27     _Dobot = Dobot
28     return True

```

Listing 3: Lazy-Import-Caches und `_try_import_ros()`

6 Kinematisches Modell

6.1 Koordinatenkonventionen (pydobot)

Die Bibliothek `pydobot` liest vom Dobot Magician über das serielle Protokoll (Kommando-ID 10) die aktuelle Pose: $(x, y, z, r, j_1, j_2, j_3, j_4)$. Dabei gilt folgende Konvention:

j_1 : Basis-Rotation (Yaw, Z-Achse) [Grad] (1)

j_2 : Oberarm-Pitch, absoluter Weltwinkel von Horizontal [Grad] (2)

j_3 : Unterarm-Pitch, absoluter Weltwinkel von Horizontal [Grad] (3)

j_4 : Servo-gesteuerte Endeffektordrehung (Wristgelenk) [Grad] (4)

Durch den Parallelogramm-Mechanismus des Dobot Magician gilt näherungsweise $j_3 \approx 0^\circ$ (Unterarm bleibt in Weltkoordinaten horizontal), sofern das Seilzug- und Zahnradsystem korrekt eingestellt ist.

6.2 Isaac-Sim-Gelenkstruktur

Das USD-Modell unter `/World/magician/base_link` definiert vier aktive Revolute-Gelenke:

DOF-Index	USD-Name	Kinematische Bedeutung	Physischer Bereich
0	<code>joint_1</code>	Basis-Yaw	$\pm 180^\circ$
1	<code>joint_2</code>	Schulter-Pitch	$0^\circ - 90^\circ$
2	<code>joint_5</code>	Parallelogramm-Kompensation	$0^\circ - 120^\circ$
3	<code>joint_6</code>	Endeffektor-Vertikalisierung	$\pm 180^\circ$

6.3 Einheitenstrategie

Alle internen Berechnungen verwenden **Grad**. Radiant-Konvertierung findet ausschließlich an den API-Grenzen statt:

Kontext	Einheit	Konvertierung
Interne Kinematik	Grad	–
DC-API <code>set_dof_state.pos</code>	Radian	<code>math.radians()</code>
DC-API <code>set_dof_position_target</code>	Radian	<code>math.radians()</code>
ROS2 <code>JointState.position</code>	Radian	<code>math.radians()</code>
UI-Dashboard-Anzeige	Grad	–

6.4 Berechnungsformeln

6.4.1 joint_1 – Gierwinkel mit festem Offset

Der Gierwinkel j_1 des Dobot wird mit einem hardcodierten Offset von $\Delta_{\text{Yaw}} = -45^\circ$ korrigiert, um die Ausrichtung der Simulationsachse gegenüber dem realen Roboter auszugleichen:

$$\theta_1 = j_1 + \Delta_{\text{Yaw}} = j_1 - 45^\circ \quad (5)$$

Der Wert ist als Modul-Konstante `J1_YAW_OFFSET_DEG = -45.0` in `extension.py` festgelegt.

6.4.2 joint_2 – Direkte Übernahme

$$\theta_2 = j_2 \quad [^\circ] \quad (6)$$

6.4.3 joint_5 – Parallelogramm-Kompensation

Der Parallelogramm-Mechanismus hält `link_5` in Weltkoordinaten horizontal. j_3 und j_2 sind beide absolute Weltwinkel; `joint_5` muss ihre Differenz kodieren:

$$\theta_5 = j_3 - j_2 \quad [^\circ] \quad (7)$$

```

1 # Parallelogramm-Kompensation: Unterarm in Weltlage halten
2 pos_j5 = j3 - j2    # Grad

```

Listing 4: Berechnung `joint_5` in `update_step()`

6.4.4 joint_6 – Endeffektor-Vertikalisierung

Anforderung: `link_6` (Saugnapf-Träger) soll stets senkrecht nach unten zeigen, unabhängig von der Armkonfiguration.

Herleitung: Die Summe aller Pitch-Rotationen entlang der kinematischen Kette muss in Weltkoordinaten 90° ergeben:

$$\theta_2 + \theta_5 + \theta_6 = 90^\circ \quad (8)$$

Einsetzen von Gleichung (7):

$$\begin{aligned} j_2 + (j_3 - j_2) + \theta_6 &= 90^\circ \\ j_3 + \theta_6 &= 90^\circ \end{aligned}$$

Berücksichtigung der Servo-Verdrehung j_4 :

$$\boxed{\theta_6 = 90^\circ - j_3 - j_4} \quad (9)$$

Wichtiger Hinweis: Der Term j_2 tritt in Gleichung (9) *nicht* auf – er kürzt sich heraus. Zwei Entwicklungsfehler wurden behoben:

1. $\text{pos_j6} = j_2 + j_3 - j_4$ ließ θ_6 mit der Schulterposition mitlaufen → Endeffektor kippte bei angehobenem Arm nach oben.
2. Eine Mischung von $\frac{\pi}{2}$ (Radiant) und Grad-Werten in einer einzigen Formel führte zu Einheitenfehlern.

```
1 # Kette: j2 + (j3-j2) + j6 = 90 => j6 = 90 - j3 - j4
2 # j2 darf hier NICHT auftreten
3 pos_j6 = 90.0 - j3 - j4 # Grad
```

Listing 5: Korrekte Berechnung joint_6 in update_step()

6.5 Vollständige Kinematik in update_step()

```
1 pose = self.device.pose() # (x, y, z, r, j1, j2, j3, j4)
2 if pose is not None and len(pose) >= 8:
3     x, y, z, r = pose[0], pose[1], pose[2], pose[3]
4     j1, j2, j3, j4 = pose[4], pose[5], pose[6], pose[7]
5
6     # Gierwinkel-Offset: Simulationsachse gegenueber Realroboter
7     j1 = j1 + J1_YAW_OFFSET_DEG # -45 Grad (hardcoded)
8
9     # Parallelogramm: Unterarm in Weltlage halten
10    pos_j5 = j3 - j2
11    # Endeffector vertikal: j2+(j3-j2)+j6=90 => j6=90-j3-j4
12    pos_j6 = 90.0 - j3 - j4
13
14    joints_deg = [j1, j2, pos_j5, pos_j6]
15    self.last_valid_positions = joints_deg
16    self.last_pose = {'x': x, 'y': y, 'z': z, 'r': r, 'j1': j1}
17
18    # ROS2-Konvention: Radiant
19    msg.position = [math.radians(v) for v in joints_deg]
20    self.publisher_.publish(msg)
```

Listing 6: Kinematik-Kern in update_step() (Ausschnitt aus extension.py)

7 Klasse DobotROSNode

7.1 Initialisierung (__init__)

Der Konstruktor führt in dieser Reihenfolge aus:

1. Lazy-Imports prüfen; bei Fehler `RuntimeError` mit beschreibender Meldung.
2. ROS2-Kontext initialisieren (`rclpy.init()`), idempotent durch `rclpy.ok()`-Guard).
3. ROS2-Node `dobot_extension_bridge` erstellen.
4. JointState-Publisher auf `/joint_states` anlegen (Queue-Tiefe 10).
5. Serielle Verbindung via `pydobot.Dobot(port=com_port)`.
6. Fallback-Puffer `last_valid_positions = [0.0, 0.0, 0.0, 0.0]`.

7.2 Methoden-Referenz

Methode	Beschreibung
<code>update_step()</code>	Liest Pose vom Dobot, berechnet Gelenkwinkel, publiziert <code>JointState</code> , ruft <code>rclpy.spin_once(timeout=0)</code> auf (nicht-blockierend). Gibt (<code>pose_dict</code> , <code>joints_deg_list</code>) zurück. Bei Fehler: Fallback auf <code>last_valid_positions</code> .
<code>set_suction(active)</code>	Schaltet Vakuumgreifer SW1 (Kommando ID 62). Fallback-Liste über mehrere pydobot-Methodennamen (<code>suck</code> , <code>set_end_effector_suction</code> , <code>set_vacuum_gripper</code>).
<code>move_vertical(delta_mm)</code>	Liest aktuelle Pose, addiert <code>delta_mm</code> auf Z, sendet <code>move_to</code> . Positiv = aufwärts.
<code>jog(dx, dy)</code>	Horizontaler Versatz in der X-Y-Ebene. Liest aktuelle Pose, addiert (<code>dx</code> , <code>dy</code>) in mm.
<code>go_home()</code>	Fährt zu HOME_X/Y/Z/R aus <code>constants.py</code> .
<code>set_servo_gp3(angle_deg)</code>	RC-Servo an GP3-PWM1 via Dobot-Rohprotokoll (Kommandos ID 130 + ID 132). Winkel $[0^\circ, 180^\circ]$, linearer Duty-Cycle. Details: Abschnitt 8.
<code>play_sequence(poses)</code>	Einfache Sequenz-Wiedergabe ohne Sicherheits-Anfahrt (interne Fallback-Methode; im Betrieb durch <code>_play_sequence_thread</code> übernommen).
<code>stop_sequence()</code>	Leert Dobot-interne Queue via <code>_set_queued_cmd_stop_exec</code> , <code>_set_queued_cmd_clear</code> , <code>_set_queued_cmd_start_exec</code> .
<code>shutdown()</code>	Schließt COM-Port via <code>device.close()</code> , zerstört ROS2-Node via <code>destroy_node()</code> .

7.3 Fehlerbehandlung und Robustheit

Kommunikationsunterbrechung: `update_step()` fängt alle Ausnahmen auf. Bei Fehler werden die Werte aus `last_valid_positions` verwendet – das Simulationsmodell *bleibt in der letzten bekannten Stellung*.

pydobot-Versionsinkompatibilität:

```

1 for name, args in [
2     ('suck', (active,)),
3     ('set_end_effector_suction', (active,)),
4     ('set_vacuum_gripper', (1 if active else 0,)),
5 ]:
6     fn = getattr(self.device, name, None)
7     if callable(fn):
8         return fn(*args)
9 raise RuntimeError('Keine Saugbefehl-Methode in pydobot gefunden.')
```

Listing 7: Suction-Fallback-Mechanismus in `set_suction()`

8 Servo-GP3-Ansteuerung

8.1 Hardware-Hintergrund

Der Dobot Magician bietet einen GP3-Port (General Purpose Port 3) mit einem PWM-fähigen Steuerausgang PWM1 (IO-Adresse 15, alternativ PWM2 = Adresse 16). Über diesen lässt sich ein Standard-RC-Servo (Trägerfrequenz 50 Hz, Pulsbreite 1–2 ms) ansteuern.

8.2 Duty-Cycle-Berechnung

Der Duty-Cycle wird linear zwischen $D_{\min} = 0,05$ (1 ms-Puls, entspricht 0°) und $D_{\max} = 0,10$ (2 ms-Puls, entspricht 180°) interpoliert:

$$D(\alpha) = D_{\min} + \frac{\alpha}{180^\circ} \cdot (D_{\max} - D_{\min}), \quad \alpha \in [0^\circ, 180^\circ] \quad (10)$$

8.3 Protokoll-Implementierung

Da pydobot keine High-Level-API für GP3-PWM bereitstellt, werden zwei Dobot-Rohprotokoll-Kommandos direkt gesendet:

```

1 angle_deg = max(0.0, min(180.0, float(angle_deg)))
2 duty = SERVO_MIN_DUTY + (angle_deg / 180.0) * \
3     (SERVO_MAX_DUTY - SERVO_MIN_DUTY)
4
5 # Kommando 130: Pin 15 auf PWM-Modus (Multiplex-Typ 2)
6 mux = _Msg()
7 mux.id = 130
8 mux.ctrl = 0x03 # Schreiben + in Queue
9 mux.params = bytearray()
10 mux.params.extend(struct.pack('B', GP3_PWM_ADDRESS)) # 15
11 mux.params.extend(struct.pack('B', 2)) # Typ: PWM
12 self.device._send_command(mux)
13
14 # Kommando 132: Frequenz und Duty-Cycle setzen
15 pwm = _Msg()
16 pwm.id = 132
17 pwm.ctrl = 0x03
18 pwm.params = bytearray()
19 pwm.params.extend(struct.pack('B', GP3_PWM_ADDRESS))
20 pwm.params.extend(struct.pack('f', SERVO_FREQ_HZ)) # 50.0 Hz
21 pwm.params.extend(struct.pack('f', duty))
22 self.device._send_command(pwm)

```

Listing 8: set_servo_gp3() – Rohprotokoll

8.4 J1-Weltwinkel-Kompensation

Beim Setzen des Servo-Winkels wird der *Weltwinkel* gespeichert:

$$\alpha_{\text{Welt}} = \alpha_{\text{Servo}} + j_1 \quad (11)$$

Bei jeder Fahrt in der Teach-in-Wiedergabe (`_servo_j1_compensate()`) wird der Servo-Winkel rückgerechnet:

$$\alpha_{\text{Servo,neu}} = \max(0^\circ, \min(180^\circ, \alpha_{\text{Welt}} - j_{1,\text{aktuell}})) \quad (12)$$

Damit bleibt die Raumorientierung eines gehaltenen Objekts bei wechselnder Basisrotation konstant.

9 Dynamic-Control-Kopplung

9.1 Initialisierungsproblem

Die Isaac-Sim-Dynamic-Control-API benötigt eine *laufende* Physik-Simulation. Vor dem Drücken von *Play* liefert `get_articulation()` `INVALID_HANDLE`.

9.2 Lazy-Initialisierung

```

1 if self._art_init_pending and self._dc is None:
2     import omni.timeline
3     if omni.timeline.get_timeline_interface().is_playing():
4         robot_path = self._robot_path_input.model \
5             .get_value_as_string().strip()
6         self._init_articulation(robot_path, silent=True)
7     if self._dc is not None:
8         self._art_init_pending = False

```

Listing 9: Lazy DC-Init in `_on_update()`

9.3 DOF-Handles abrufen

```

1 joint_names = ['joint_1', 'joint_2', 'joint_5', 'joint_6']
2 for jname in joint_names:
3     dof = dc.find_articulation_dof(art, jname)
4     self._dof_handles.append(
5         dof if dof != dc_mod.INVALID_HANDLE else None
6     )
7 found = sum(1 for d in self._dof_handles if d is not None)
8 # Status: "Modell verbunden: 4/4 DOFs gefunden."

```

Listing 10: DOF-Handle-Abruf in `_init_articulation()`

9.4 Gelenkwinkel setzen – `_drive_joints()`

Pro Gelenk werden zwei komplementäre DC-Aufrufe verwendet:

```

1 for dof_handle, angle_deg in zip(self._dof_handles, joints_deg):
2     if dof_handle is None:
3         continue
4     rad = math.radians(float(angle_deg)) # Grad -> Radian
5
6     # 1) Kinematischen Zustand direkt setzen
7     state = self._dc_mod.DofState()
8     state.pos = rad
9     state.vel = 0.0
10    self._dc.set_dof_state(dof_handle, state, self._dc_mod.STATE_POS)
11
12    # 2) PhysX-Drive-Ziel setzen (Pflicht: [-2pi, 2pi])
13    self._dc.set_dof_position_target(dof_handle, rad)

```

Listing 11: `_drive_joints()` – doppelter DC-Aufruf

`set_dof_position_target()` erzwingt $[-2\pi, 2\pi]$: `PxArticulationJoint::setDriveTarget()`

only supports target angle in range $[-2\pi, 2\pi]$. Die Kinematik-Formeln halten alle Ausgaben innerhalb dieses Bereichs.

9.5 Artikulations-USD-Pfad

Die `ArticulationRootAPI` liegt beim importierten Dobot-Modell auf `/World/magician/base_link` – *nicht* auf dem Elternprim `/World/magician`. Der vorausgefüllte Standardwert im UI ist entsprechend gesetzt.

10 Teach-in-System (Pick & Place)

10.1 Datenstruktur der Wegpunkte

Jeder Wegpunkt ist ein 5-Tupel: $(x_i, y_i, z_i, r_i, s_i) \in \mathbb{R}^4 \times \{0, 1\}$ mit (x, y, z) in mm, r in Grad, s = Saugzustand (0 = aus, 1 = ein).

10.2 Aufzeichnung

Rec Punkt	Speichert <code>DobotROSNode.last_pose</code> + aktuellen virtuellen Saugzustand als neuen Wegpunkt. <i>Kein Hardware-Befehl</i> – der Saugzustand wird erst bei der Wiedergabe ausgeführt.
Saugen EIN/AUS	Wechselt den virtuellen Saugzustand <i>und</i> speichert automatisch einen Wegpunkt mit dem neuen Zustand (<code>_on_rec_suction_toggle_clicked()</code>). Ermöglicht präzises Einteachen von Greif- und Ablegepositionen.
Löschen	Setzt Wegpunktliste, Saugzustand und Zähler zurück.

10.3 Wiedergabe-Thread

Die Wiedergabe läuft in einem dedizierten `threading.Thread` (`daemon=True`, `Name: dobot_play`).

Begründung für Threading statt asyncio: `pydobot.wait_for_cmd()` ist eine blockierende Polling-Schleife. Im asyncio-Event-Loop würde sie den gesamten Isaac-Sim-Hauptthread einfrieren und alle Frame-Updates blockieren. Im Hintergrundthread blockiert sie nur diesen Thread; Isaac Sim läuft ungehindert weiter. `pydobot`s internes `RLock` garantiert Thread-Sicherheit der seriellen Zugriffe.

10.4 Bewegungsablauf pro Wegpunkt

```

1  APPROACH_MM = 10.0    # 1 cm Sicherheitsabstand von oben
2
3  for i, (x, y, z, r, suction_active) in enumerate(poses):
4      if self._stop_play_flag: break
5      suction_changes = (suction_active != prev_suction)
6
7      if suction_changes:
8          # 1 cm ueber Zielpunkt anfahren (kein seitliches Streifen)
9          _move_and_wait(x, y, z + APPROACH_MM, r)
10         _servo_j1_compensate()
11         _time.sleep(0.1)
12
13         # Eigentliche Zielposition anfahren
14         _move_and_wait(x, y, z, r)

```



```

15     _servo_j1_compensate()
16
17     # Saugzustand schalten (nach Positionserreichen)
18     self._ros_node.set_suction(suction_active)
19     _time.sleep(0.5)    # Druckaufbau/-abbau abwarten
20
21     if suction_changes:
22         # 1 cm sicher abheben
23         _move_and_wait(x, y, z + APPROACH_MM, r)
24         _servo_j1_compensate()
25
26     prev_suction = suction_active

```

Listing 12: Kernlogik _play_sequence_thread() (vereinfacht)

10.5 Rückkehr zur Nullposition

Nach dem letzten Wegpunkt oder nach Abbruch:

1. 10 mm über die Home-Position anfahren.
2. Zur eigentlichen Home-Position absenken.
3. Servo-Weltwinkel-Kompensation ausführen.
4. Sauger deaktivieren.

10.6 Notstopp

_on_stop_sequence_clicked() setzt _stop_play_flag = True und ruft DobotROSNode.stop_sequence() auf, das die interne Dobot-Bewegungs-Queue leert.

11 Ziel-Würfel-Tracking

11.1 Würfel erstellen (_spawn_target_cube())

1. Alten Würfel entfernen (Idempotenz).
2. Startposition aus DobotROSNode.last_pose (mm → m: Division durch 1000).
3. UsdGeom.Cube, Kantenlänge 2,5 cm, unter /World/DobotTargetCube.
4. Oranges UsdPreviewSurface-Material (diffuse RGB (1.0, 0.45, 0.0), Opacity 0,9) unter /World/DobotTargetCubeMat.
5. Tracking-Thread starten.

11.2 Tracking-Schleife (_tracking_loop_thread())

```

1 while not self._stop_tracking_flag and self._ros_node:
2     pos = self._get_cube_world_position()    # m (USD-Stage)
3     if pos is not None:
4         x = pos[0] * 1000.0    # m -> mm
5         y = pos[1] * 1000.0
6         z = pos[2] * 1000.0
7         r = self._ros_node.last_pose.get('r', 0.0)
8         move_fn(x, y, z, r)    # Bewegungsbefehl an Dobot

```

```
9 _time.sleep(1.0 / max(0.1, self._tracking_rate_hz))
```

Listing 13: Tracking-Thread-Kernlogik

Die Weltposition wird via `UsdGeom.Xformable.ComputeLocalToWorldTransform()` exakt ausgelesen (inkl. Elterntransformationen). Tracking-Frequenz: 0,5–10,0 Hz, einstellbar per Slider.

11.3 Würfel entfernen (`_despawn_target_cube()`)

Stop-Flag setzen → Thread mit `join(timeout=1 s)` abwarten → USD-Prims aus Stage entfernen → UI zurücksetzen.

12 Klasse DobotBridgeExtension

12.1 Zustandsvariablen

Variable	Bedeutung
<code>_ros_node</code>	Aktive <code>DobotROSNode</code> -Instanz (oder <code>None</code>)
<code>_update_sub</code>	Update-Event-Subscription (automatisch gefeuert)
<code>_suction_active</code>	Aktueller Sauger-Status
<code>_servo_angle</code>	Aktueller Servo-Winkel in Grad (Startwert 90°)
<code>_servo_world_angle</code>	Servo-Weltwinkel für J1-Kompensation
<code>_recorded_poses</code>	Liste aller aufgezeichneten Wegpunkte
<code>_rec_suction_state</code>	Virtueller Saugzustand (nur Aufzeichnung)
<code>_play_thread</code>	Hintergrundthread der Sequenz-Wiedergabe
<code>_stop_play_flag</code>	Abbruchsignal für Wiedergabe-Thread
<code>_cube_spawned</code>	Würfel aktuell in Stage
<code>_tracking_thread</code>	Hintergrundthread für Würfel-Tracking
<code>_stop_tracking_flag</code>	Abbruchsignal für Tracking-Thread
<code>_tracking_rate_hz</code>	Tracking-Sendefrequenz (Standard: 2,0 Hz)
<code>_dc</code>	DC-Interface nach erfolgreicher Initialisierung
<code>_dc_mod</code>	DC-Modul-Referenz für <code>DofState</code> -Konstruktor
<code>_art_handle</code>	Artikulations-Handle
<code>_dof_handles</code>	Liste von DOF-Handles [<code>j1, j2, j5, j6</code>]
<code>_art_init_pending</code>	True nach Connect, False nach erfolgreicher DC-Init
<code>_dock_task</code>	Async-Docking-Task

12.2 Lebenszyklusmethoden

<code>on_startup(ext_id)</code>	Alle Zustandsvariablen initialisieren; UI-Fenster (490×590 px) aufbauen; Docking-Task starten.
<code>_dock_window_deferred()</code>	Async-Coroutine: 2 Frames warten, Fenster neben Property-Panel docken via <code>ui.DockPosition.SAME</code> . Zwei Frames notwendig, da Fenster erst nach erstem Render im Workspace-Index erscheint.
<code>_on_update(event)</code>	Frame-Callback: DC-Lazy-Init, Dobot-Polling, Gelenkwinkeltransfer, Dashboard-Update.
<code>on_shutdown()</code>	Dock-Task canceln (verhindert “extension object is still alive”-Warnung), <code>_cleanup()</code> aufrufen, Fenster zerstören.

12.3 Frame-Update-Ablauf (_on_update())

```

1 def _on_update(self, event):
2     if not self._ros_node:
3         return
4
5     # 1. Lazy DC-Initialisierung
6     if self._art_init_pending and self._dc is None:
7         import omni.timeline
8         if omni.timeline.get_timeline_interface().is_playing():
9             robot_path = self._robot_path_input.model \
10                 .get_value_as_string().strip()
11             self._init_articulation(robot_path, silent=True)
12             if self._dc is not None:
13                 self._art_init_pending = False
14
15     # 2. Dobot-Daten lesen und ROS2 publizieren
16     pose, joints = self._ros_node.update_step()
17
18     # 3. DC-Kopplung (nur wenn Simulation laeuft)
19     import omni.timeline
20     if omni.timeline.get_timeline_interface().is_playing():
21         self._drive_joints(joints)
22
23     # 4. Dashboard aktualisieren
24     self._update_display(pose, joints)

```

Listing 14: _on_update() – vollständiger Ablauf

12.4 Ressourcenfreigabe (_cleanup())

1. Stop-Flag Wiedergabe-Thread setzen; join(timeout=1 s).
2. Stop-Flag Tracking-Thread setzen; join(timeout=1 s).
3. Würfel-Prims aus USD-Stage entfernen.
4. Update-Subscription freigeben.
5. DC-Interface und Handles loslassen.
6. DobotROSNode.shutdown() aufrufen (COM-Port schließen, ROS2-Node zerstören).

13 UI-Fenster

13.1 Tab 0: Steuerung

UI-Element	Funktion
COM-Port-Feld	Serieller Port (Standard: COM3)
Connect-Button	Verbindungsaufbau; zeigt nach Erfolg grün ●
Disconnect-Button	Verbindungsstrennung und UI-Reset
Verbindungsstatus (●)	Rot = getrennt, grün = verbunden
Robot-USD-Feld	USD-Pfad der Artikulations-Root in der Szene; Standard <code>/World/magician/base_link</code>
Install packages	Installiert <code>pydobot</code> und <code>pyserial</code> via <code>omni.kit.pipapi</code>
Suction ON/OFF	Vakuumgreifer umschalten
Up / Down 10 mm	Vertikalbewegung ± 10 mm
X $\pm$ / Y $\pm$ 5 mm	Horizontaler JOG ± 5 mm
Servo GP3 $\pm 10^\circ$	Servo-Winkel erhöhen / verringern; Winkel-Anzeige-Label
Würfel spawnen / entfernen	Ziel-Würfel ein-/ausblenden
Rate-Slider (0,5–10 Hz)	Tracking-Frequenz konfigurieren
Rate-Label	Zeigt aktuelle Tracking-Frequenz

13.2 Tab 1: Pick & Place

UI-Element	Funktion
Go Home	Roboter zur Nullposition fahren; Button-Label zeigt $x/y/z$ -Koordinaten
Servo GP3 $\pm 10^\circ$	Identisch zu Tab 0
Saugen (Aufn.) EIN/AUS	Virtueller Sauger-Toggle für Aufzeichnung; schaltet Zustand um <i>und</i> speichert automatisch Wegpunkt
Rec Punkt	Aktuellen Wegpunkt manuell aufzeichnen
n Pkt.	Zähler der gespeicherten Wegpunkte
Löschen	Alle Wegpunkte und Saugzustand zurücksetzen
Play	Teach-in-Sequenz im Hintergrundthread starten
Stop	Laufende Sequenz abbrechen und Dobot-Queue leeren
Scrollbare Waypoint-Liste	Zeigt alle Punkte mit Koordinaten und Saugzustand; Saugpunkte in Grün

13.3 Status-Dashboard (immer sichtbar)

Element	Beschreibung
Status-Label	Farbkodierter Betriebsstatus: grün = OK / Verbunden, gelb = Pending / Warnung, rot = Fehler
ROS-Topic-Label	ROS topic: /joint_states
Pose-Label	Kartesische Position: Pose: x=... y=... z=... r=...
Joints-Label	Gelenkwinkel in Grad: Joints [°]: j1=... j2=... j5=... j6=...

14 Threading-Modell

Thread	Name	Aufgabe
Hauptthread	(Isaac Sim)	UI-Callbacks, Frame-Updates (<code>_on_update</code>), DC-API-Aufrufe
Wiedergabe	<code>dobot_play</code>	Blockierende <code>wait_for_cmd()</code> -Schleife
Tracking	<code>dobot_cube_track</code>	Periodische <code>move_to</code> -Befehle für Würfel

Thread-Sicherheit: pydobots internes `RLock` serialisiert alle seriellen Zugriffe. Statusschreibzugriffe aus Hintergrundthreads (`_set_status()`) sind durch den GIL (Global Interpreter Lock) von CPython atomisch.

Graceful Shutdown: Beide Stop-Flags werden in `_cleanup()` gesetzt; danach `join(timeout=1 s)` pro Thread.

15 ROS2-Integration

15.1 Node-Konfiguration

Parameter	Wert
Node-Name	<code>dobot_extension_bridge</code>
Topic	<code>/joint_states</code>
Nachrichtentyp	<code>sensor_msgs/msg/JointState</code>
Queue-Tiefe	10
Publish-Frequenz	≈ 60 Hz (Frame-Rate-gekoppelt)
RMW	<code>rmw_fastrtps_cpp</code> (Fast DDS)
ROS2-Distro	Humble (eingebettet in Isaac-Sim-Bridge)

15.2 Nachrichtenstruktur

```

1 msg.header.stamp = self._node.get_clock().now().to_msg()
2 msg.name         = ['joint_1', 'joint_2', 'joint_5', 'joint_6']
3 msg.position     = [math.radians(j1),          # Radiant (ROS2-Konvention)
4                   math.radians(j2),
5                   math.radians(pos_j5),
6                   math.radians(pos_j6)]
7 # velocity und effort bleiben leer (optionale Felder)

```

Listing 15: JointState-Nachricht in `update_step()`

15.3 ROS2-Kontext-Verwaltung

`rclpy.init()` wird genau einmal aufgerufen (`rclpy.ok()`-Guard verhindert Mehrfach-Init). `rclpy.spin_once(timeout_sec=0.0)` am Ende von `update_step()` verarbeitet eingehende Nachrichten nicht-blockierend.

16 Konstanten (`constants.py`)

16.1 Farbwerte (ARGB-Hex)

Konstante	Hexwert	Verwendung
CLR_BG_DARK	0xFF0A0F1A	Sehr dunkler Hintergrund
CLR_BG_MID	0xFF101828	Aktives Panel / Toolbar
CLR_ACCENT	0xFF4A9EFF	Info / Akzentfarbe
CLR_GREEN	0xFF4ADE80	Erfolg / Verbunden
CLR_RED	0xFFEF6B6B	Fehler / Getrennt
CLR_YELLOW	0xFFE0B040	Warnung / Pending
CLR_TEXT	0xFFD0D8E8	Primärtext
CLR_TEXT_DIM	0xFF607090	Sekundärtext
CLR_BORDER	0xFF1A2840	Trennlinien

16.2 Nullposition und Servo

Konstante	Wert	Beschreibung
HOME_X	−3,7 mm	Nullposition X
HOME_Y	227,8 mm	Nullposition Y
HOME_Z	66,1 mm	Nullposition Z
HOME_R	88,4°	Nullposition Wristdrehung
GP3_PWM_ADDRESS	15	IO-Adresse PWM1 am GP3-Port
SERVO_FREQ_HZ	50,0 Hz	RC-Servo-Trägerfrequenz
SERVO_MIN_DUTY	0,05	1 ms-Puls = 0
SERVO_MAX_DUTY	0,10	2 ms-Puls = 180

16.3 USD-Pfade

Konstante	Pfad
ROBOT_USD_PATH	/World/magician/base_link

17 URDF-Robotermodell (`load_dobot.urdf`)

Das URDF (Unified Robot Description Format) beschreibt die kinematische Struktur des Dobot Magician für den Isaac-Sim-Import.

17.1 Gelenkstruktur

Gelenk	Typ	Achse	Limits
joint_1	revolute	Z (Yaw)	$\pm 180^\circ$, ± 10 rad/s
joint_2	revolute	Y (Schulter)	0° – 90° , ± 10 rad/s
joint_5	revolute	Y (Parallelogramm)	0° – 120° , ± 10 rad/s
joint_6	revolute	Y (Endeffektor)	$\pm 180^\circ$, ± 10 rad/s
joint_suction	fixed	–	–

17.2 Körper (Links)

Link	Masse
base_link	1,0 kg
link_1 – link_4	je 0,3 kg
link_5, link_6	0,1 kg
link_suction	0,05 kg

18 Installation und Inbetriebnahme

18.1 Erstinstallation

1. **Isaac Sim starten:** C:\NVIDIA_Isaac_Sim\isaac-sim-dobot.bat aufrufen (setzt OMNI_KIT_EXTENSION_ automatisch).
2. **Extension aktivieren:** *Window* → *Extensions*; nach `dobot_ros` suchen und aktivieren.
3. **Pakete installieren:** Im Extension-Panel *Install packages* klicken. Dies installiert `pydobot` und `pyserial` in der Isaac-Sim-Python-Umgebung via `omni.kit.pipapi`. Danach Extension deaktivieren und neu aktivieren.

18.2 Betrieb Schritt für Schritt

1. Szene mit Dobot-Magician-Modell laden.
2. ArticulationRootAPI auf `base_link` prüfen (*Property* → *Physics* → *Articulation Root*).
3. COM-Port eingeben (Standard: COM3) und **Connect** klicken. Status: *Verbunden – warte auf Play...*
4. **Play** drücken. Status: *Modell verbunden: 4/4 DOFs gefunden*.
5. Simulationsmodell folgt echtem Roboter in Echtzeit.

18.3 Teach-in-Workflow

1. Tab *Pick & Place* wählen.
2. Roboter manuell in Ausgangsposition fahren (Jog-Tasten).
3. *Rec Punkt* klicken.
4. An Greifpunkt: *Saugen (Aufn.)* auf EIN – automatisch Waypoint gespeichert.
5. Schritte 2–4 für alle Punkte wiederholen.

6. *Play* starten; Sequenz läuft mit Sicherheits-Anfahrt.
7. *Stop* bricht jederzeit ab.

19 Bekannte Einschränkungen

- **ROS2 eingebettet:** Die Extension verwendet die in Isaac Sim eingebettete ROS2 Humble Bridge. Eine separate ROS2-Installation ist nicht erforderlich, jedoch auch nicht kompatibel ohne Anpassung der Bibliothekspfade.
- **DC-API vor Play:** `get_articulation()` vor dem ersten Play-Ereignis erzeugt eine Warnung. Durch Lazy-Init tritt diese nur einmal auf.
- **Suction-Methoden-Varianz:** Die Fallback-Liste deckt bekannte pydobot-Versionen ab; bei zukünftigen Versionen ggf. erweitern.
- **Servo-Zustand nach Reconnect:** Nach Disconnect/Reconnect startet der Servo-Winkel bei 90° – der tatsächliche Hardware-Winkel kann abweichen.
- **Keine Greifer-Rückmeldung:** Saugbefehle werden als Fire-and-Forget gesendet. Ob ein Objekt tatsächlich gegriffen ist, kann ohne Drucksensor nicht verifiziert werden.
- **Physikalisch ungültige Winkel:** Winkel außerhalb $[-2\pi, 2\pi]$ führen zum PhysX-Fehler in `set_dof_position_target()`. Die Kinematik-Formeln halten alle Ausgaben im gültigen Bereich.

20 Entwicklungsverlauf

Commit	Meilenstein
1611913	Initiales Repository-Setup
d2744a9	ROS2-Bridge-Integration und Merge
1ed673f	Schrittmotor-Kopplung funktionsfähig
d099a83	Live/Sim-Umschaltung implementiert
c919311	Vakuumgreifer-Ansteuerung funktionsfähig
7ddfb27	Squeeze-/Pick-Logik funktionsfähig
a95763a	Deckel-Maze-Bewegung implementiert
76b73a6	MQTT-Integration für Deckel/Maze
72ec561	MQTT für Digital-State, -Move und Binary-Move
1ef5eca	Stabiler Arbeitsstand
7471a6f	Vollständig kommentierter Digital-Twin
685be33	Fehlerbehebungen (Kinematik-Formeln)
5cd2262	Pick-&-Place vollständig funktionsfähig (aktuell)